

Trabalho Prático 1: Alinhamento Estratégico de Carteiras

Relatório Técnico (README.pdf) | Abordagem: Busca em Profundidade (DFS) com Árvore Binária Implícita | Linguagem: Java 17+

1. Descrição e Modelagem da Solução Desenvolvida

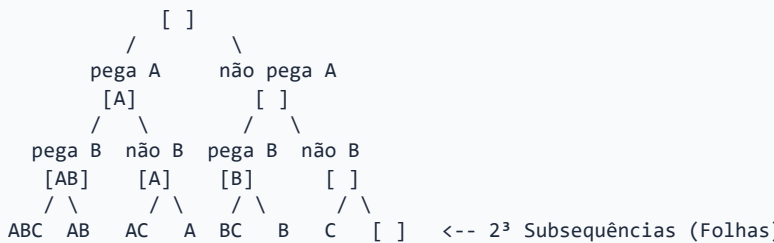
O objetivo do trabalho é determinar o **Núcleo de Consenso Ordenado** entre carteiras recomendadas emitidas por duas instituições de análise (**Corretora A** e **Corretora B**). Esse alinhamento exige identificar a maior sequência de ações em comum preservando rigorosamente a ordem relativa de recomendação em ambos os relatórios.

1.1. Modelagem Autoral: A Árvore Binária Implícita de Decisões

Diferente das abordagens tabulares clássicas de Programação Dinâmica, esta solução modela o espaço de estados do problema através do percurso sobre uma **árvore binária de decisões implícita**. Para cada ativo $A[i]$ da Corretora A, o algoritmo ramifica em duas decisões booleanas excludentes:

- Ramo Esquerdo ("Pega o ativo"):** Inclui o ativo $A[i]$ na subsequência candidata e avança para a avaliação de $A[i+1]$.
- Ramo Direito ("Não pega o ativo"):** Descarta o ativo $A[i]$ e avança para $A[i+1]$ sem alterar a subsequência corrente.

Para uma entrada simplificada com 3 ativos $[A, B, C]$, a estrutura binária completa percorrida pelo algoritmo é representada a seguir:



1.2. Percurso em Profundidade (DFS) com Backtracking

O algoritmo percorre a árvore por meio de **Busca em Profundidade** (*Depth-First Search* — *DFS*). O estado do caminho é mantido dinamicamente em uma lista:

- Ao descer pelo ramo esquerdo ("pega"), adiciona-se o ticker atual ao final da lista candidata.
- Ao retornar da recursão, executa-se o **Backtracking** (`caminhoAtual.remove(caminhoAtual.size() - 1)` em $O(1)$), restaurando o estado anterior para viabilizar a exploração pura do ramo direito ("não pega").
- Ao alcançar uma folha (índice igual a M), tem-se uma subsequência candidata completa gerada a partir de A.

1.3. Validação Linear de Subsequência na Corretora B

Ao atingir uma folha com tamanho estritamente superior ao da melhor solução registrada até o momento (`caminhoAtual.size() > melhorSolucao.size()`), aciona-se a verificação com a **Corretora B**.

O método `ehSubsequencia` realiza uma varredura linear com ponteiro sobre o vetor **B**: cada ticker da sequência candidata é procurado em **B** em posição posterior à ocorrência do anterior. Sendo validado com sucesso, a solução ótima é atualizada.

1.4. Otimização por Poda de Viabilidade (Branch and Bound)

Para evitar a expansão de ramificações que matematicamente não podem superar o consenso corrente, implementou-se uma **poda por cota superior**:

CrITÉrio de Poda (Bounding Function):

Se `caminhoAtual.size() + (M - indice) ≤ melhorSolucao.size()`, o ramo atual é descartado.

Se o número de ativos já acumulados somado a *todos* os ativos restantes em **A** for insuficiente para bater o recorde atual, o nó e toda a sua subárvore são abortados imediatamente.

2. Análise de Complexidade de Tempo e Espaço

2.1. Complexidade de Tempo

- Espaço de Estados da Árvore:** Uma árvore binária com altura M possui 2^M nós-folha e $2^{M+1} - 1$ nós no total.

- **Custo por Nó e Folha:** Cada decisão na árvore consome tempo constante $O(1)$. Nas folhas promissoras, a validação linear em B percorre no máximo N elementos em tempo $O(N)$.
- **Pior Caso Teórico:** Quando a poda não elimina nós precocemente (ex.: sequências sem nenhuma interseção), o algoritmo visita todas as folhas, totalizando $O(2^M \cdot N)$.
- **Desempenho com Poda:** Na presença de alinhamentos comuns nos primeiros ramos da DFS, a cota superior rapidamente elimina extensas subárvores, viabilizando execução célere para conjuntos de teste reais.

Etapa do Algoritmo	Complexidade de Tempo	Descrição
Leitura dos Dados (STDIN)	$O(M + N)$	Leitura linear dos vetores das duas corretoras.
Geração da Árvore DFS	$O(2^M)$	Bifurcação binária ("pega" / "não pega") para os M ativos de A.
Validação em B (Folhas Promissoras)	$O(N)$	Varredura gulosa com ponteiro sobre o vetor da Corretora B.
Escrita da Saída (STDOUT)	$O(K)$	Impressão da quantidade e dos K tickers alinhados.
COMPLEXIDADE DE TEMPO TOTAL (Pior Caso)	$O(2^M \times N)$	

2.2. Complexidade de Espaço (Memória Auxiliar)

- **Pilha de Execução (Recursão):** A profundidade da recursão corresponde à altura da árvore (M chamadas empilhadas), resultando em consumo espacial $O(M)$.
- **Memória Auxiliar:** As listas `caminhoAtual` e `melhorSolucao` mantêm até M referências, requerendo $O(M)$ de espaço.
- **Complexidade Espacial Total:** $O(M + N)$ para armazenar a entrada e apenas $O(M)$ de espaço auxiliar (sem alocação de matrizes bidimensionais).

3. Descrição do Uso de IA (Inteligência Artificial)

O uso de inteligência artificial durante o desenvolvimento deste trabalho prático compreendeu:

1. **Modelagem da Árvore Binária Implícita:** Apoio na formulação da árvore de decisões binárias ("pega" / "não pega") para geração sistemática das subsequências via DFS.
2. **Implementação do Mecanismo de Poda:** Dedução formal da cota superior de viabilidade (*bounding function*) para descarte de subárvores subótimas.
3. **Automação de Testes e Validação:** Estruturação de rotinas de teste para verificação estrita contra os exemplos oficiais do documento do trabalho.
4. **Elaboração dos Relatórios Técnicos:** Diagramação e formatação da documentação técnica acadêmica em formatos HTML e PDF.